Unidad 1 / Escenario 2

Lectura fundamental

Estándares de codificación de *software*

Contenido

1 Codificación general

2 Sintaxis HTML

3 Patrones, estandarización y sintaxis html

Palabras clave: proceso personal de *software*, estándar HTML, W3C, Codificación, formato PSP.

Una vez registrado el tiempo es necesario identificar el rendimiento asociado a la codificación. Una de las métricas que ayudan a establecer el rendimiento es el número de líneas de código, para determinar el número de horas es importante estandarizar la manera cómo se construye *software*, a continuación, se ilustran dos tipos de codificación para el mismo problema de saber cuál de los dos números es mayor.

```
<!DOCTYPE html>
<html>
<head>
<title>Ejemplo</title>
</head>
<body>
        EL MAYOR ES:
        <p id="SALIDA"></p>
    <script language="JavaScript" type="text/JavaScript">
        var numero1=30;
        var numero2=40;
        if (numero1<numero2){
            document.getElementById("SALIDA").innerHTML = numero2;
        }else{
            document.getElementById("SALIDA").innerHTML = numero1;
        }
    </script>
</body>
</html>
```

Figura 1. codificación normal validación del mayor entre dos números

Fuente: elaboración propia

```
<!DOCTYPE html>
<html>
<head><title>Ejemplo</title></head>
<body>
        EL MAYOR ES:
        <p id="SALIDA"></p>
    <script language="JavaScript" type="text/JavaScript">
        var numero1=30;var numero2=40;
        if (numero1<numero2){
            document.getElementById("SALIDA").innerHTML = numero2;
        }else{document.getElementById("SALIDA").innerHTML =
        numero1;}
    </script>
</body>
</html>
```

Figura 2. Codificación normal validación del mayor entre dos números

Fuente: elaboración propia

Las dos codificaciones producen el mismo resultado, y tienen exactamente el mismo número de instrucciones, sin embargo, el número de líneas de código son 19 para el primer caso y 14 para el segundo.

Si no existe un estándar de codificación, es imposible determinar con claridad cuál es el rendimiento del programador. Es importante determinar en este caso, cuáles serían las mejores prácticas y que debemos tener en cuenta como línea de código y qué no.

1. Codificación general

Para cualquier tipo de lenguaje existen algunas características que se deben tener en cuenta, a esto se le conoce como codificación general. No todos los lenguajes necesitan llaves "Signo ortográfico auxiliar en forma de ballesta ({ })" sin embargo para la mayoría de lenguajes nos surge la pregunta: ¿Las llaves (línea 16 Ilustración 1) cuentan como línea de código?, la respuesta es no, si bien le dan mejor presentación al código y debemos.

Existen algunas herramientas en línea que pueden ayudarnos a verificar incluso la codificación general. Una vez desplegado un sitio web podemos validarlo a través de W3C en su página web, si se desea ver un ejemplo pude visitar el sitio oficial[1] y tomar cualquier URL o dirección corporativa como ejemplo para evaluarlo.

1.1. Comentarios:

¿En qué momento los comentarios cuentan como línea de código?, si bien los comentarios son fundamentales a la hora de revisar, modificar o "refactorizar" un código y demandan un tiempo de trabajo y construcción, sobre todo si el autor los redacta de manera detallada, no se tendrán en cuenta como líneas de código.

Es necesario manejar un estándar o un lenguaje común y dependiendo del interés de medición se tendrán en cuenta o no. Vale aclarar que, si el programador desea incluir los comentarios en el análisis de su desarrollo personal, deberá realizar el cálculo de esfuerzo de manera aislada, independiente al código fuente.

1 https://www.w3schools.com

1.2. Indentación

La indentación, sangrando o tabulación es desplazar el código un número considerable de espacios a la derecha de la margen izquierda. Es importante definir el tipo de indentación, que se requiere manejar en el documento, pero más que eso, establecer en que momento la legibilidad del código debe estar por encima de la identación.

Como estándar para la construcción del código en el proyecto, los encabezados HTML (<html>, <head>,<body>) no deben tener indentación y el espaciado para el resto de la codificación no debe superar media hoja.

Hoy en día algunos editores traen las reglas de para la indentación que mejoran la apariencia del código.

1.3. Nominación

El último aspecto a considerar, en este apartado, en la codificación general es la nominación. Se trata de la manera en que nombramos a cada uno de los elementos o funciones dentro de nuestro código, a continuación, se muestra una tabla de los tipos más usados.

Tabla 1. Estructura código HTML

Título	Ejemplo	Nominación
Encabezado	**HTML:** <!DOCTYPE html>	Debe estar en mayúsculas sostenidas la primera palabra después del símbolo de admiración y va en todos los archivos HTML.
Etiqueta	**HTML:** <a> <head></head>	Los nombres de las etiquetas y atributos siempre se escriben en minúsculas.

Comentarios	**Java Script:**
	// Esto es un comentario.
	//This is a comment.
	HTML:
	<!—Esto es un comentario -->
	<!-- this is a comment -->
		De acuerdo a las reglas gramaticales de la lengua en que está escrita el front o parte visual del código. Ingles en el caso de que existan varios idiomas.
		Los comentarios largos en HTML tienen la misma estructura que los cortos (Abren y cierran igual).

Variables	**Java script:** var nombre_variable= 2;	Como el lenguaje es sensible a mayúsculas y minúsculas es mejor solo manejar minúsculas.

Fuente: elaboración propia

2. Sintaxis HTML

La sintaxis es la forma en que debe construirse el código, a diferencia de la semántica que le da el sentido a lo que escribimos. La sintaxis solo se preocupa por la manera en que se escriben las estructuras de código para que se procese un resultado específico. A continuación, se presenta una estructura de código (sintaxis) y al lado de ella el resultado especifico, para probar cualquiera de los *scripts* o códigos que se presentan en la tabla, debe usar la siguiente estructura de código base o plantilla:

```
1  <!DOCTYPE html>
2  <html>
3  <body>
4  <!-- Inserte aquí el código de prueba. -->
5  </body>
6  </html>
```

Figura 3. Estructura base de código html

Fuente: elaboración propia

2.1. Tipos de Listas

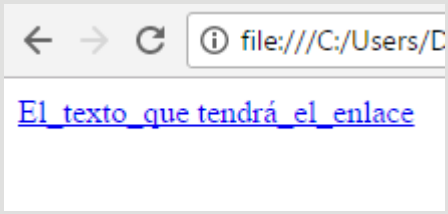
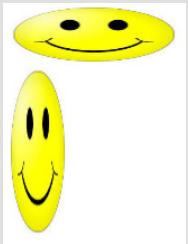
Tabla 2. Tipos de listas

  - Análisis - Diseño - Desarrollo 	• Análisis • Diseño • Desarrollo
  - Análisis - Diseño - Desarrollo 	1. Análisis 2. Diseño 3. Desarrollo

Fuente: elaboración propia

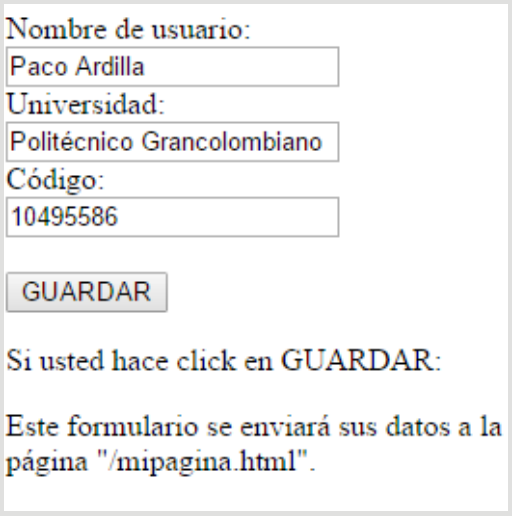
2.2. Tablas, enlaces, imágenes

Tabla 3. Tablas, enlaces, imágenes

<table> <tr> <th>Requerimiento</th> <th>Tiempo planeado</th> <th>Tiempo real</th> </tr> <tr> <td>Tabla</td> <td>20</td> <td>10</td> </tr> <tr> <td>Botón</td> <td>10</td> <td>5</td> </tr> </table>	<table><tr><th>Requerimiento</th><th>Tiempo planeado</th><th>Tiempo real</th></tr><tr><td>Tabla</td><td>20</td><td>10</td></tr><tr><td>Botón</td><td>10</td><td>5</td></tr></table>
 El_texto_que tendrá_el_enlace 	


2.3. Formularios

Tabla 4. Formularios

<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" /> <body> <form action="/mipagina.html"> Nombre de usuario: <input type="text" name="Nombre" value="Paco Ardilla"> Universidad: <input type="text" name="universidad" value="Politécnico Grancolombiano"> Código: <input type="text" name="codigo" value="10495586"> <input type="submit" value="GUARDAR"> </form> Si usted hace click en GUARDAR: Este formulario se enviará sus datos a la página "/mipagina.html". 	

Fuente: elaboración propia

3. Patrones, estandarización y sintaxis html

¿Qué es un patrón?

Un patrón es la solución a un problema recurrente en el mundo del *software*. Para que sea considerado patrón debe describir como mínimo las siguientes características:

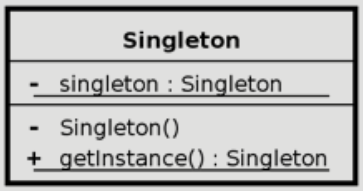
Nombre, problema, solución, código, diagrama de clases, un ejemplo de esto es el patrón singleton:

Tabla 4. Patrón singleton

Nombre:	Singleton
Problema:	Cada vez que se instancia una clase de este tipo se tienen que definir los atributos de la clase a pesar de que siempre sea el mismo.

Solución:	var user = {
	firstName: 'John',
	lastName: 'Doe',
	sayName: function() {
	return this.firstName + ' ' + this.lastName;
	}
	};

Código:	public final class Singleton {
	private static final Singleton INSTANCE = new Singleton();
	private Singleton() {}
	public static Singleton getInstance() {
	return INSTANCE;
	}
	}

Diagrama de clases	

Fuente: elaboración propia.

Son usados generalmente para **estandarizar** soluciones a problemas que se presentan con frecuencia en el desarrollo del software, la esencia del patrón aplica independiente del lenguaje de programación que se utilice.

¿Cómo cambia el concepto en HTML?

La palabra patrón es muy usada en el mundo del desarrollo de software sin embargo si hablamos de código HTML el término hace referencia a los **validadores de campo** que se utilizan en la mayoría de los lenguajes.

Los validadores de campo utilizan expresiones regulares que tienen una sintaxis especial pero que no es propiamente codificación:

¿Cómo se usan los patrones o validadores de campo en los formularios html?

Se utilizan con el elemento input type y permiten asegurarse de que la información que se ingresa en el formulario tenga el formato requerido, en el código de ejemplo que se presenta a continuación, el patrón se asegura (valida) que se utilicen seis o más caracteres:

```
<form action=”/Pagina_destino.html”>
Contraseña: <input type=”password” name=”pw” pattern=”.{6,}” title=”Utilice seis o más caracteres”>
 <input type=”submit”>
</form>
```

Referencias bibliográficas

Humphrey, W. S. (2002). Personal Software Process (PSP). *PERSONAL SOFTWARE PROCESS (PSP). Encyclopedia Of Software Engineering, Volume 2*, 948-961.

Ministerio del interior. (01 de 05 de 2017). *Dirección Nacional de Derechos de Autor, unidad administrativa especial.* Obtenido de Registro de soporte lógico(software): http://derechodeautor.gov.co/software

Software Engineering Institute (SEI). (01 de 05 de 2017). *Software Engineering Institute.* Obtenido de Carnegie Mellon University: http://www.sei.cmu.edu/about/

W3C. (05 de 02 de 2017). *Markup Validation Service.* Obtenido de https://validator.w3.org/

w3Schools. (01 de 05 de 2017). *w3schools.com.* Obtenido de w3schools.com: https://www.w3schools.com

INFORMACIÓN TÉCNICA

FACULTAD DE
INGENIERÍA, DISEÑO
E INNOVACIÓN

Módulo: Proceso de *Software* Personal PSP

Unidad 1: Introducción al proceso de *software* personal

Escenario 2: Estándares de codificación de software

Autor: Diego Iván Oliveros Acosta

Asesor Pedagógico: Jeiner Velandia

Diseñador Gráfico: Kelly Yohana Valencia Forero

Asistente: Ginna Paola Quiroga Espinosa

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.